

RED5 AHU

Controller Commissioning Guide

Embedded Delta controller · Web interface on port **5001**

This guide takes a new controller from factory state to a running RED5 AHU system. Steps 1–10 start the web server, steps 11–13 install the application, and steps 14–16 start telemetry collection.

You never paste a large script into enteliWEB. Each program object holds a single line that loads the real code from the controller's own storage. This is deliberate — pasting the full program fails on current hardware, as explained in step 8.

What you need before starting: access to enteliWEB, the controller's IP address, the `red5_bundle.zip` supplied with your system, and network access to an existing commissioned RED5 controller if you have one.

The Debug Console runs one line at a time. Every command in this guide is written as a single line for that reason. If your text editor or PDF viewer wraps one across several lines when you copy it, re-copy it — a multi-line paste will be rejected.

PART A — START THE WEB SERVER

1 Install and network the controller

Rack and power the controller, and connect it to the network segment enteliWEB can reach. Note its IP address — you will need it from step 10 onward.

2 Discover the controller in enteliWEB

Run a device discovery and confirm the new controller appears with the expected device instance and name before continuing.

3 Set the controller time

Correct the controller's date, time, and **time zone**.

Do this before anything else. Telemetry snapshots, band scheduling, and weather history are all stored against timestamps. A controller commissioned on the wrong clock produces trend data that looks corrupted later and is tedious to correct.

Check the time zone specifically. Controllers have been found running on UTC while the site was hours ahead. Everything still works, so nothing warns you — the discrepancy only surfaces later in historical trends.

4

Enable PG Python on the device

Open the controller's **DEV** (device) object and enable **PG Python**. Python programs cannot be created or run until this is switched on.

5

Create a program object named “app” in Python mode

Create a new **PG** (Program) object, name it **app**, and set its program mode to **Python** using the **Mode** button in the editor toolbar.

Program mode is the most common commissioning mistake. If the mode is not Python, the object still accepts and saves your text, but every attempt to run it fails with **Programming Error** — even a single `print()` statement. See *Troubleshooting*.

Confirm the object was created correctly before continuing. In the Debug Console:

```
import os; print(sorted(os.listdir('/root/scripts')))
```

You should see `['app.py']`. An empty list means the program mode is wrong. Fix it now rather than troubleshooting later.

The name matters. The controller firmware removes any Python file that is not a registered enteliWEB object. Naming the object `app` is what allows the program to persist.

6

Install the required modules

Install these two modules, and only these two:

```
pip install flask
pip install flask_cors
```

Do not install Pillow. It is not required. The system detects its absence and skips the one optional feature that uses it (image thumbnail resizing). Pillow and its dependencies consume tens of megabytes of the controller's limited storage for no practical benefit.

Check available space:

```
import shutil; u=shutil.disk_usage('/root'); print('FREE %.2f MB' %
(u.free/1048576))
```

7

Let the installations finish

Allow each `pip install` to complete before continuing. Starting the program while a module is still installing produces a misleading `ImportError`.

8

Copy the program file onto the controller

The main program is roughly 90 KB and **must not be pasted into the program object**. It is copied to the controller's storage instead, where there is no size limit.

If you already have a commissioned RED5 controller on the network, the new controller can download the file directly from it. Run this in the Debug Console of the *new* controller, substituting the working controller's IP address:

```
import urllib.request, os; os.makedirs('/root/data/pgpy', exist_ok=True);
urllib.request.urlretrieve('http://<working-controller-
ip>:5001/assets/app_canonical_c2.py.txt', '/root/data/pgpy/app_main.py');
print('bytes:', os.path.getsize('/root/data/pgpy/app_main.py'))
```

You should see `bytes: 89565`. A smaller number means the download was interrupted — delete the file and repeat rather than continuing.

Commissioning your first controller? With no existing controller to copy from, request the program file from your system integrator and make it available over HTTP from any computer on the same network, then use that address in the command above.

Why the file is not pasted. On current hardware a paste this large is rejected with `QERR_CODE_NO_SPACE`, even though the controller has plenty of free storage. The limit applies to how much program text a single object can hold. The object may also keep only the first portion of the text, so the save appears to succeed and then fails when run.

9

Enter the loader line and run

The complete contents of the **app** program object is this one line:

```
exec(compile(open('/root/data/pgpy/app_main.py').read(),
'/root/data/pgpy/app_main.py', 'exec'))
```

Save — it should commit immediately, with no loading delay — then set the object to **Run**.

You will not need to edit this object again. Future software updates replace the program file in storage; this loader line stays exactly as it is, and updates arrive through the bundle upload in step 11.

This object does not restart automatically. If the program ever stops, it stays stopped until someone starts it again in enteliWEB. This matters in step 12 and after any power interruption.

10

Check the output window

On a new controller the application is not installed yet, so the program starts in recovery mode and tells you where to go next. Expect output like this:

```
[service-discovery] no *_service.py files found in ['/root/data/pgpy',  
'/root/scripts']  
[bootstrap-fallback] No upload service registered - installing emergency  
bootstrap so the operator can deploy upload_service.py via /update  
  
* Controller Online at: http://<controller-ip>:5001  
* Update page at: http://<controller-ip>:5001/update
```

The last two lines confirm a healthy start. The `no *_service.py files found` message is expected at this stage and disappears once the application is installed.

PART B — INSTALL THE APPLICATION

11

Upload the bundle

Browse to `http://<controller-ip>:5001/`. You are redirected to `/update` and shown the red **Emergency Bootstrap** page. Upload `red5_bundle.zip` there.

The password field accepts any entry. The standard bundle is not encrypted, so the value is never checked — the form simply will not submit while the field is empty. A real key is required only for encrypted bundles created by the Download/Replicate feature.

The upload installs the application, its configuration, and the telemetry collector. Your program object from step 9 is protected and cannot be overwritten by an upload.

12

Stop and start the “app” object

The application loads its components once at startup, so nothing you just uploaded is active yet. Stop the **app** program object in enteliWEB, then start it again.

The output window should now show one `registered OK` line per component, and the `bootstrap-fallback` message should be gone.

13

Verify the installation

Check these in order, in a browser. All addresses are relative to `http://<controller-ip>:5001`.

ADDRESS	EXPECTED RESULT
<code>/api/services</code>	Every component reports <code>OK</code> . Any <code>FAILED</code> entry includes the reason.
<code>/api/health</code>	<code>{"ok": true, "version": "1.9.0"}</code>
<code>/</code>	The access page, no longer redirecting to <code>/update</code> .
<code>/dashboard</code>	The dashboard renders.
<code>/mobile</code>	The phone view renders.

PART C — START TELEMETRY COLLECTION

The collector reads live BACnet points and writes the data the dashboard displays. It requires its own program object: the controller only provides BACnet access to registered enteliWEB objects, so this work cannot run inside the `app` object. The collector program file was already installed by the bundle in step 11 — you are only creating the object that runs it.

14

Create a program object named “collector” in Python mode

Exactly as in step 5: create a new **PG** object, name it **collector**, and set its program mode to **Python** with the **Mode** button. Then confirm:

```
import os; print(sorted(os.listdir('/root/scripts')))
```

You should now see `['app.py', 'collector.py']`. If `collector.py` is missing, the program mode is not set to Python and nothing you enter will run.

Do not try to create this file yourself. Files placed in the scripts folder by any other means appear to save and are then removed by the controller firmware. Only creating the object in enteliWEB works.

15

Enter the loader line and run

The complete contents of the **collector** object is this one line:

```
import sys; sys.argv=['collector.py'];
exec(compile(open('/root/data/pgpy/collector.py').read(),
'/root/data/pgpy/collector.py', 'exec'))
```

Save, then set the object to **Run**. Expect output like:

```
Red5 Telemetry Collector v1.2 starting...
Config path: /root/data/configs/collector_config.json
Mock mode: False
Telemetry output: /root/data/configs/telemetry.json
Discovered 0 AHU groups
Poll interval: 5s
```

Enter the line exactly as shown. Both halves are required: the first prepares the program's start-up options, and omitting it causes an immediate error. Do not add anything to the end — the collector starts itself.

16

Confirm data is being collected

Browse to `http://<controller-ip>:5001/api/telemetry-status`. You are looking for `age_seconds` below about 10 and `stale: false`. A large `age_seconds` value that keeps increasing means the collector is not running.

Discovered 0 AHU groups is normal at this point. It means no equipment has been mapped yet. Use the equipment mapper to define your AHUs and VAVs; live values appear once that is complete. This is configuration, not a fault.

TROUBLESHOOTING — “PROGRAMMING ERROR” EVERY TIME YOU RUN

Symptom: the object saves your text without complaint, but running it immediately shows `Program_Change : Programming Error` and **Halted**.

Test it by replacing the entire program with one harmless line and running that:

```
print('hello')
```

If even that fails, the object is not set to Python mode. Confirm with:

```
import os; print(sorted(os.listdir('/root/scripts')))
```

A missing entry for your object confirms it. Correct the **Mode** setting rather than changing the program text — no edit to the code can fix this.

TROUBLESHOOTING — “NO_SPACE” WHEN SAVING

Symptom: saving a large program raises `QERR_CODE_NO_SPACE`. This is a limit on how much program text one object can store, not a shortage of controller storage — it occurs with tens of megabytes free. The object may also keep only part of your text, so the save appears to succeed and then fails when run.

The solution is the approach used throughout this guide: keep each object to a single loader line and hold the program itself in storage, as in steps 8, 9 and 15. To confirm storage is healthy:

```
import shutil; u=shutil.disk_usage('/root'); print('total %.1f MB / used %.1f MB / FREE  
%.2f MB' % (u.total/1048576, u.used/1048576, u.free/1048576))
```

If space really is low, remove Pillow with `pip uninstall -y Pillow` — keep only `flask` and `flask_cors`.

TROUBLESHOOTING — “AUTHENTICATION REQUIRED” WHEN UPLOADING

This is not the bundle password, which is not checked for standard bundles. It means user authentication has been switched on for this controller. Sign in as `admin` using the system master key as the password, then retry the upload.

If you do not have the master key, it is stored on the controller and can be displayed from the Debug Console:

```
print(open('/root/data/master_key.txt').read().strip())
```

COMMISSIONING FURTHER CONTROLLERS

Once this controller is running it can supply the program file to the next one, so you do not need the original file again. Use this address as the download source in step 8:

```
http://<this-controller-ip>:5001/assets/app_canonical_c2.py.txt
```